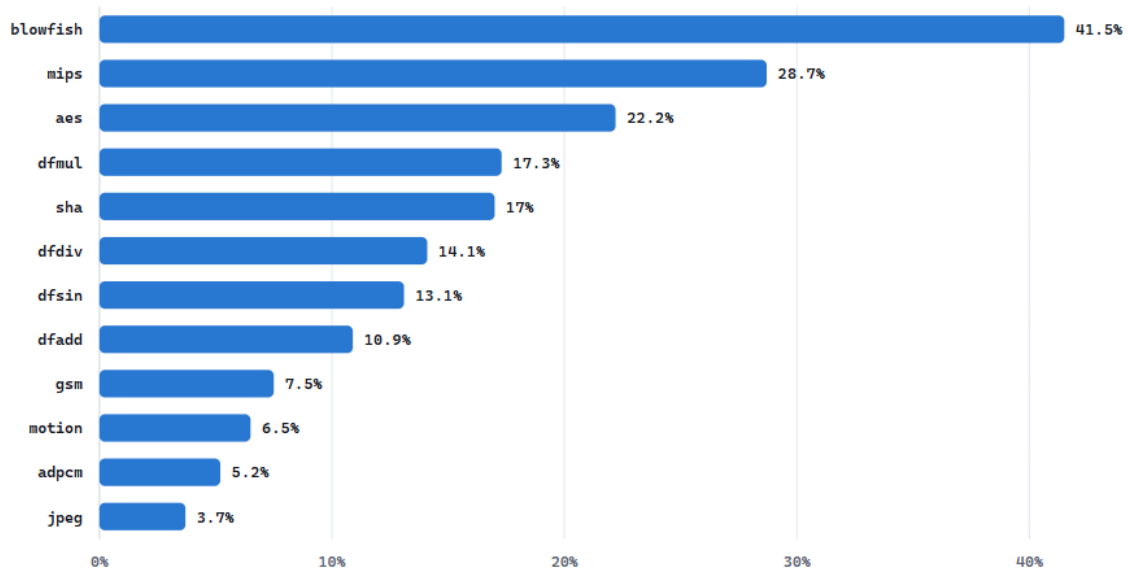


260729

[CHSTONE 케이스 분석]

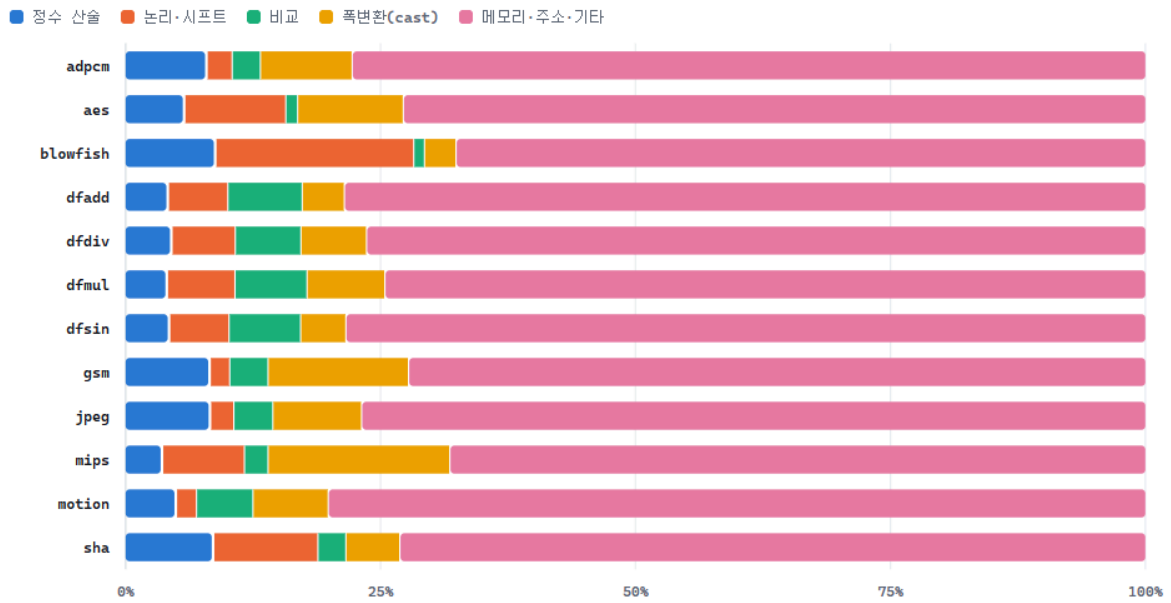
Wasted %

wasted % = (선언폭 - 유효폭) / 선언폭

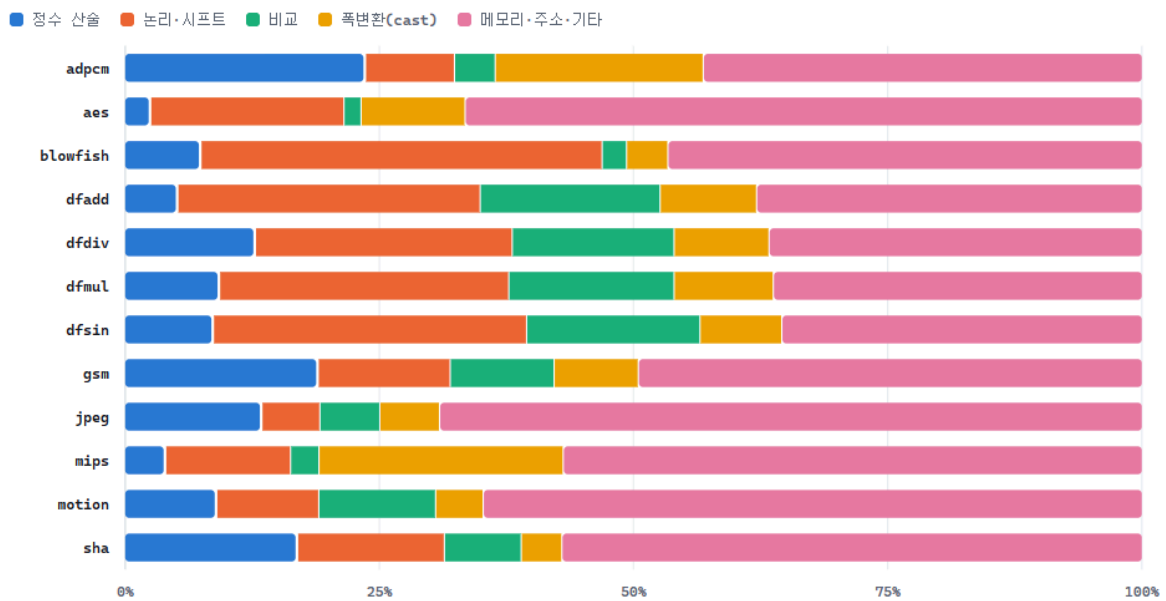


모든 벤치마크의 datapath 정수 연산은 대부분 i32/i64로 선언

Operation Mix(OO)



Operation Mix(O2)

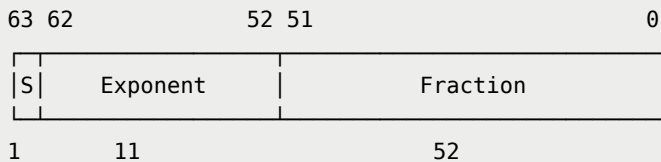


▼ Softfloat

```
static float64
addFloat64Sigs (float64 a, float64 b, flag zSign)
{
    int16 aExp, bExp, zExp;
    bits64 aSig, bSig, zSig;
    int16 expDiff;

    // 생략 //
}
```

IEEE-754 double 구조



exponent는 사실상 11비트로 표현되는데... 여기서는 16비트로 두었음.

마찬가지로 expDiff도 범위가 -2047 ~ +2047 니까 12비트로 표현 가능함.

Significands도 52비트인데 64비트로 표현되고 있음.

Exp는 Shift 1회, AND 1회 Sig는 AND 1회로 추출함.

expDiff를 계산하는 이유는 두 실수의 exp가 다르면 같은 exp로 맞춰준 다음에 계산을 해야하기 때문이다.

정규화된 normal number는 항상 앞자리가 1이다. 그래서.. 굳이 1을 저장하지 않는다.

예를 들어서 1.101101010... 뭐 이런식이면 101101010... 이부분만 저장한다.

이 생략된 1을 Hidden Bit 또는 Implicit Bit 뭐 이런식으로 부른다.

subnormal number의 경우는 앞자리가 항상 0이다.

그래서 bExp == 0 이면 hiddenBit를 따로 계산안한다.

1.

JSA 방식에서 산술연산에 대해 보수적으로 carry가 발생한다고 가정함.

Value Range를 쓰면 실제로 carry가 발생하는지 안하는지 판단이 가능해짐.

정수 산술 연산에 대한 최적화로 Value Range가 효과를 볼것으로 기대됨.

2.

Floating Point 가 없어서 이것에 대한 최적화 여지는 찾기 어려워보임.

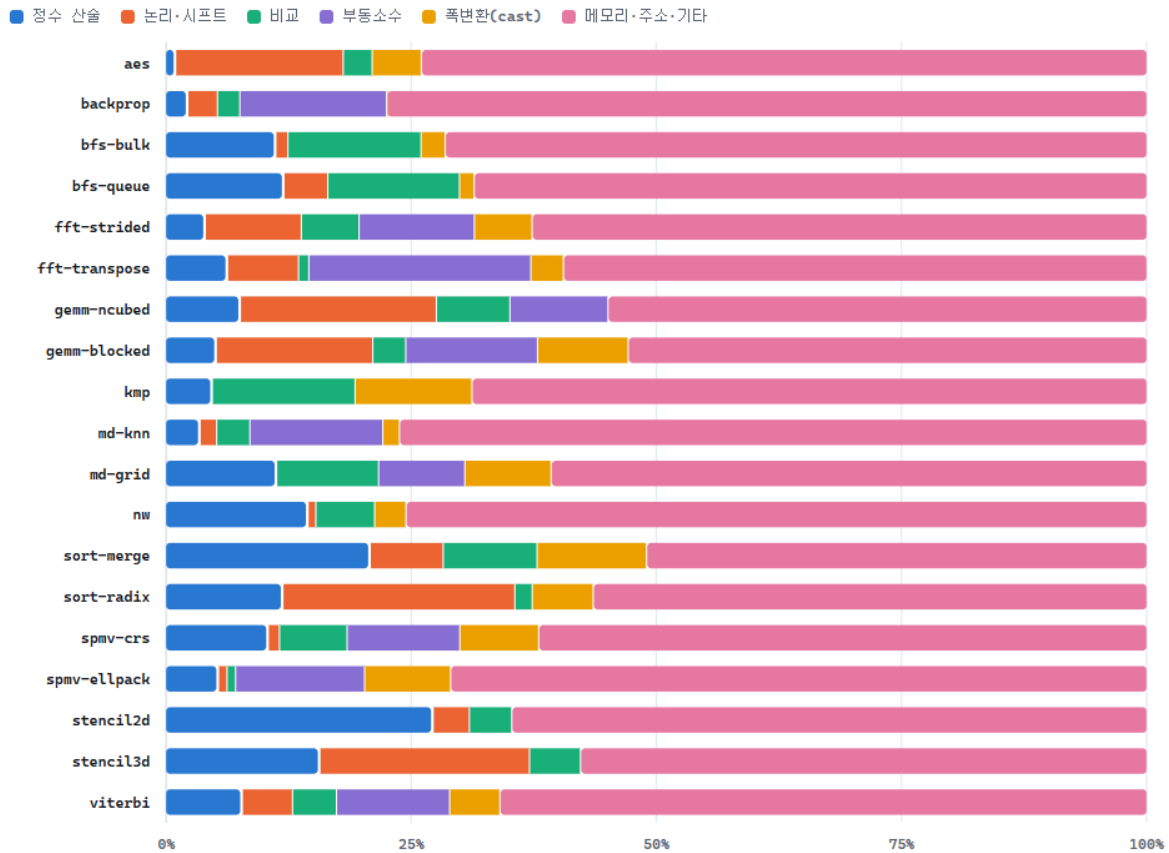
그래서 MachSuite 벤치마크 후술

3.

Floating Point 에서 exponential 비트를 value range analysis를 통해 최적화.

Exp는 정밀도 희생 없이 가능할 것으로 보임.

MachSuite



근데 이거도 aes · bfs-bulk · bfs-queue · kmp · nw · sort-merge · sort-radix · stencil2d · stencil3d는 부동소수 연산이 없음.

Floating Point

지수부랑 가수부

가수부에 대한 최적화는 정밀도에 영향을 줄 수 있음.

지수부는 범위에 대한 기존의 Value Range Analysis를 채용하면 될 수 있을것 같음.

Automating Customisation of Floating-Point Designs - 2002

Floating-Point Bitwidth Analysis via Automatic Differentiation - 2002

Unifying Bit-Width Optimisation for Fixed/Floating Point - 2004

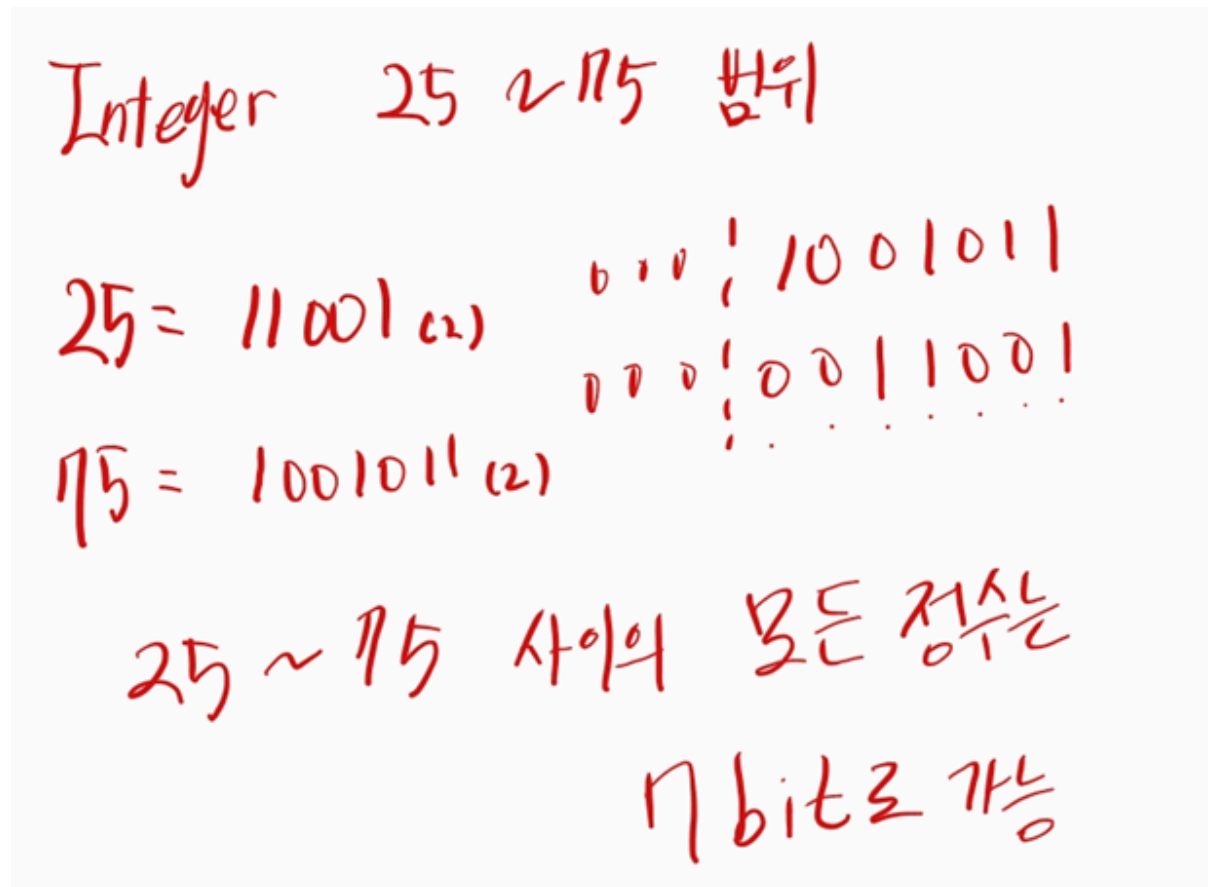
MiniBit / MiniBit+

Significands 를 Value Range Analysis를 통해 Optimize 하면 발생할 수 있는 문제
범위가 좁다는 의미 → 비트를 적게 사용해도 된다는 의미가 아님.

Integer랑 다르게 비트를 적게 사용해도 된다는 것은 정확도를 더 낮게 가져가도 된다는 의미로 해석할 수 있음.

Value Range를 부동소수점에 그대로 쓰기 애매한 이유

Integer



Floating Point

$$5.25 = 101.01_{(2)}$$

$$5.75 = 101.11_{(2)}$$

5.25 ~ 5.75 사이의 실수 5.375

$$5.375 = 101.011_{(2)}$$

$$5.25 \text{ 가수부} = 0101000000 \dots$$

$$5.375 \text{ 가수부} = 0101100000 \dots$$

$$5.75 \text{ 가수부} = 0111000000 \dots$$

사이 값이 정확히 일정한 비트를 쓴다는 보장이 없음.

정수는 비트 수가 값의 크기라고 치환하여 생각할 수 있지만 부동소수점은 비트 수가 정밀도, 세밀도와 관련됨.

정밀도 분석

프로그램에서 부동 소수점 자료형의 정밀도의 영향을 분석해서 가수부의 비트 수를 줄이는 방식...

- Floating-point bitwidth analysis via automatic differentiation

- OptiFEX: A Framework for Exploring Area-Efficient Floating Point Expressions on FPGAs With Optimized Exponent/Mantissa Widths
- Wordlength Optimization for Custom Floating-Point Systems

민감도나 error eval, user constraints를 기반으로 mantisa width 결정

Resource Binding 분석

IR 기반으로 연산 종류별 개수와 비트폭

근데 실제 바인딩은 스케줄링에 따라 달라지기 때문에 vitis로 직접 확인할 필요가 있음.

bit operation과 ALU 다양성의 trade off의 최적점을 찾는 알고리즘 구상?

```
=====
Analyzing: adpcm.base.ll
=====

IR file      : adpcm.base.ll
Basic blocks : 59
Analyzed ops : 171

RESOURCE      OPS      WIDTHS  UNITS      UNIT WIDTHS  SHARING
WIDTH-COST
-----
INT_ADD_SUB   61      32,64    5      64,64,32,32,32  12.20
224
INT_CMP      30      32,64    2      64,32          15.00
96
INT_LOGIC     10      32,64    1      64             10.00
64
INT_MUL      33      64       5      64,64,64,64,64  6.60
320
INT_SHIFT    37      32,64    5      64,64,64,64,32  7.40
288
```

ASAP(Maximum Parallel) 리소스 스케줄링 가정해서 필요 유닛 개수